

SMART ENERGY MANAGER -

Integrated Energy Management System

*A Report submitted in partial fulfilment of
the requirements for the award of the degree of*

BACHELOR OF TECHNOLOGY (B. Tech.)

In

ELECTRICAL & COMPUTER ENGINEERING

As Project is done

By

Aditya Parashar (252305050)

Chirag Chandan (252305058)

Sahil Kumar (252305064)

Under The Guidance Of

Dr. Vijay Garg

Associate Professor, UIET, KUK



UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY

KURUKSHETRA UNIVERSITY, KURUKSHETRA- 136119

SESSION (2023-2027)

Certificate

This is to certify that the project report entitled "Smart Energy Manager – Integrated Energy Management System" is submitted by Aditya Parashar, Chirag Chandan and Sahil Kumar in partial fulfillment of the requirements of the project work under the Department of Electrical & Computer Engineering.

The project work has been carried out under the guidance of Dr. Vijay Kumar Garg and is found satisfactory for submission.

Student Signatures

Supervisor Signature

**HOD / Project
Coordinator**

Acknowledgement

We would like to express our sincere gratitude to the Department of Electrical & Computer Engineering for providing the opportunity to work on this project. We are thankful to our project guide, Dr. Vijay Kumar Garg, for guidance, suggestions, and regular support during the development of the project.

We also thank our classmates, laboratory staff, and all those who helped directly or indirectly in the completion of this project. This project helped us understand practical energy monitoring, power factor correction, protection control, relay interfacing, and Blynk mobile app-based monitoring.

Declaration

We hereby declare that this project report is prepared by us based on the practical implementation, testing, and study of the project. The work presented in this report is original to the best of our knowledge and has not been copied from any other source.

We understand that originality, discipline, timely progress, and active participation are important requirements of the departmental project work.

Names: Aditya Parashar,
Chirag Chandan,
Sahil Kumar

Signature:

Date:

Abstract

The Smart Energy Manager is a mobile app-based electrical monitoring and control system designed to measure electrical parameters, improve power factor, and protect the load during abnormal conditions. The system uses ESP32, PZEM-004T energy meter module, relay module, capacitor bank, and Blynk mobile application. It measures voltage, current, power, energy, frequency, and power factor in real time, and sends all readings to the Blynk mobile dashboard to the Blynk mobile dashboard.

- The system monitors electrical parameters in real time.
- APFC logic switches capacitor stages when power factor goes below the set value.
- It provides emergency stop, over-voltage protection, over-current protection, fault latching, and reset control.
- Blynk dashboard displays live values, trend graphs, billing analytics, low power factor alerts, and monthly energy reports.
- The project demonstrates a practical solution for smart energy monitoring and basic load protection in small electrical systems.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Methodology and Working Principle
6. Circuit Diagram
7. Hardware Requirements
8. Software Requirements
9. Circuit and Pin Connection Details
10. Blynk IoT Setup and Interface
11. ESP32 Program Logic
12. Testing and Observations
13. Results and Discussion
14. Advantages
15. Applications
16. Future Scope
17. Conclusion
18. References
19. ESP32 Program Code

1. Introduction

Electrical energy monitoring and efficient utilization are important requirements in modern domestic, commercial, and industrial systems. Many electrical loads operate at low power factor, which increases line current and losses. Manual monitoring of voltage, current, power factor, and energy consumption is not convenient for continuous operation.

The Smart Energy Manager project is developed to solve this issue by combining smart energy metering, automatic power factor correction, protection control, and mobile app-based monitoring. The system continuously measures electrical parameters and displays them on the Blynk mobile dashboard. The user can monitor the status remotely and control the system through a mobile interface.

The project uses ESP32 as the main controller because it provides wireless communication, mobile dashboard connectivity and sufficient input-output pins for sensor and relay interfacing. The PZEM-004T module is used to measure electrical parameters. The relay module controls the main supply, load switching, and capacitor bank stages for APFC operation.

2. Problem Statement

Modern electrical systems often depend on multiple independent devices and platforms for different functions, resulting in increased hardware dependency, higher complexity, and limited centralized management.

Major challenges in conventional systems include:

- **Separate systems for different functions:**

Smart meters are used for energy monitoring, APFC panels for power factor correction, MCBs/relays for protection, and separate software for analytics and supervision, leading to lack of integration.

- **Poor power factor and electrical losses:**

Low power factor can increase current draw, reduce efficiency, increase line losses, and may also lead to utility penalty charges.

- **Overcurrent and overvoltage risks:**

Fault conditions can threaten equipment safety, reduce system reliability, and cause damage or unwanted shutdowns.

- **Limited real-time monitoring and live data visibility:**

Many conventional systems do not provide continuous live tracking of electrical parameters, trends, billing status, and system performance.

- **Lack of easy remote access and modern app-based control:**

Traditional setups often lack convenient mobile monitoring, remote control, instant alerts, and user-friendly digital interfaces.

Therefore, there is a need for an integrated intelligent energy management solution capable of combining:

- ✓ Real-time Monitoring
- ✓ Automatic Power Factor Correction (APFC)
- ✓ Protection Logic
- ✓ Billing & Energy Analytics
- ✓ Notifications & Alerts
- ✓ IoT-Based Remote Supervision

The proposed Smart Energy Manager addresses these challenges by providing a unified platform for monitoring, protection, optimization, analytics, and remote control using ESP32, PZEM sensing, relay-based control, and a mobile IoT dashboard.

3. Objectives

The main objective of this project is to design a smart system that can monitor electrical parameters, improve power factor, and protect the load during unsafe conditions. This project also focuses on mobile app-based monitoring, so that the user can view live data and control the system through the Blynk mobile dashboard.

- To measure voltage, current, power, energy, frequency, and power factor in real time.
- To display live values and graphs on the Blynk Mobile dashboard.
- To switch capacitor stages automatically when power factor becomes low.
- To provide emergency stop / kill switch control for disconnecting the system.
- To protect the load during over-voltage and over-current conditions.
- To calculate energy cost and provide periodic energy report notifications.
- To create a hardware/software-based practical project suitable for electrical and computer engineering applications.

4. Scope of the Project

The scope of this project is to develop a demonstration-level smart energy management system for small AC loads. The system focuses on mobile app-based monitoring and control, relay-based control, automatic power factor correction, and basic load protection.

- This project is designed for demonstration-level smart energy management for small AC loads.
- It can monitor important electrical parameters like **voltage, current, power, energy, frequency, and power factor**.
- It provides **Mobile-based monitoring and control** through the **Blynk** mobile dashboard.
- It includes **relay-based control** for main supply, load switching, and capacitor bank switching.
- It supports **automatic power factor correction** by switching capacitor stages according to load condition.
- It provides basic protection against **over-voltage** and **over-current conditions**.
- It is suitable for educational demonstration, lab projects, and small load monitoring.

5. Methodology and Working Principle

The Smart Energy Manager works on the principle of real-time sensing, decision-making, and relay-based control. In this system, the PZEM-004T module continuously measures AC voltage, current, power, energy, frequency, and power factor. These values are sent to the ESP32 through UART serial communication. After receiving the data, ESP32 checks whether the readings are valid and within safe limits. The processed values are then sent to the Blynk mobile application for live monitoring.

- The PZEM-004T reads voltage, current, power, energy, frequency, and power factor.
- ESP32 receives these readings through UART serial communication.
- ESP32 checks safe limits, low power factor condition, auto/manual mode, and kill switch status.
- Blynk dashboard shows live values, graphs, billing analytics, relay status, and alert notifications.
- The complete system works in a continuous loop of sensing, checking, controlling, and updating.

1. Sense PZEM-004T reads voltage, current, power, energy, frequency, and power factor.	2. Decide ESP32 checks safe limits, low power factor condition, auto/manual mode, and kill switch status.
3. Control Relay module switches main supply, load path, and capacitor stages according to the decision.	4. Inform Blynk dashboard shows live values, graphs, billing analytics, and alert notifications.

5(a). APFC, Protection Logic and Overall Flow

When the system is in auto mode and no fault is present, ESP32 checks the power factor of the connected load. If the power factor falls below **0.90**, the APFC logic switches capacitor stages through relays. A delay is used between switching to avoid rapid relay operation. When the power factor improves, capacitor stages are removed gradually.

For protection, ESP32 compares voltage and current with set limits. If voltage exceeds **270 V** or current exceeds **1.1 A**, the fault is latched and relays are turned **OFF**. The system can be restarted only after voltage and current return to safe values. The emergency stop button on Blynk can also disconnect the system manually.

- If power factor goes below 0.90, capacitor stages are switched ON through relays.
- Capacitor stages are switched with delay to avoid fast relay switching.
- If power factor improves beyond the reset value, capacitor stages are removed gradually.
- If voltage exceeds 270 V, over-voltage protection trips the system.
- If current exceeds 1.1 A, over-current protection trips the system.
- Fault latching keeps the relay OFF until manual reset is done safely.
- Blynk sends live readings, graphs, alerts, billing data, and energy report to the user.

5(b). Overall Flow of Operation:

- Start ESP32 and connect it to Blynk mobile application.
- Read voltage, current, power, energy, frequency, and power factor from PZEM-004T.
- Display live values and graphs on the Blynk dashboard.
- Check over-voltage, over-current, low power factor, and emergency stop conditions.
- Switch relay and capacitor stages according to the decision.
- Send notifications and energy report to the user through the mobile application.
- Mobile app stage: Blynk displays voltage, current, power, energy, frequency, power factor, cost, relay status, and alert notifications on the mobile dashboard.
- Reset stage: After a fault, the system does not restart automatically. Manual reset is used only when voltage and current come back to safe values.

Smart Energy Manager

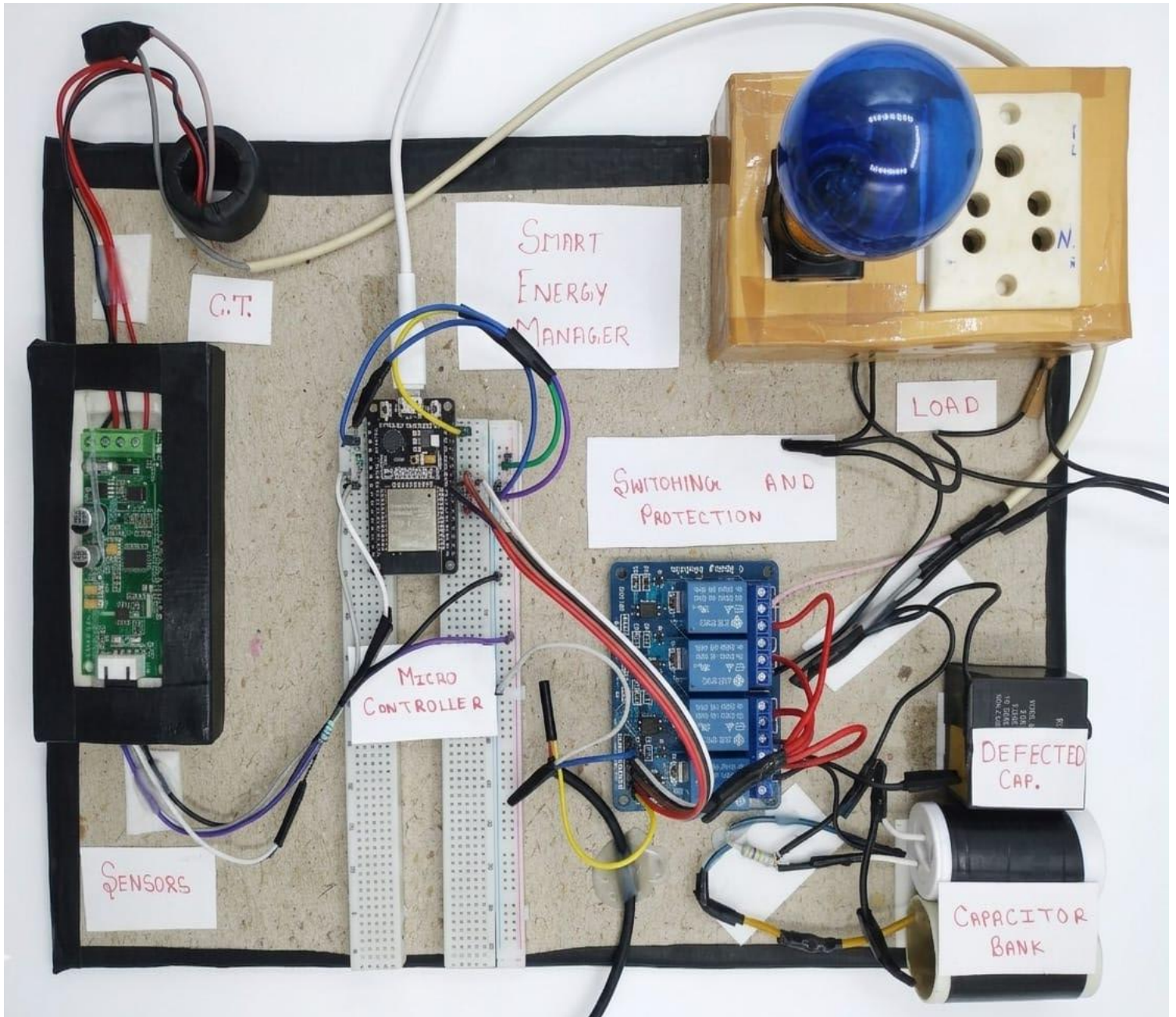


Figure 1: Real image of project (*Smart Energy Manager*)

6. Circuit Diagram

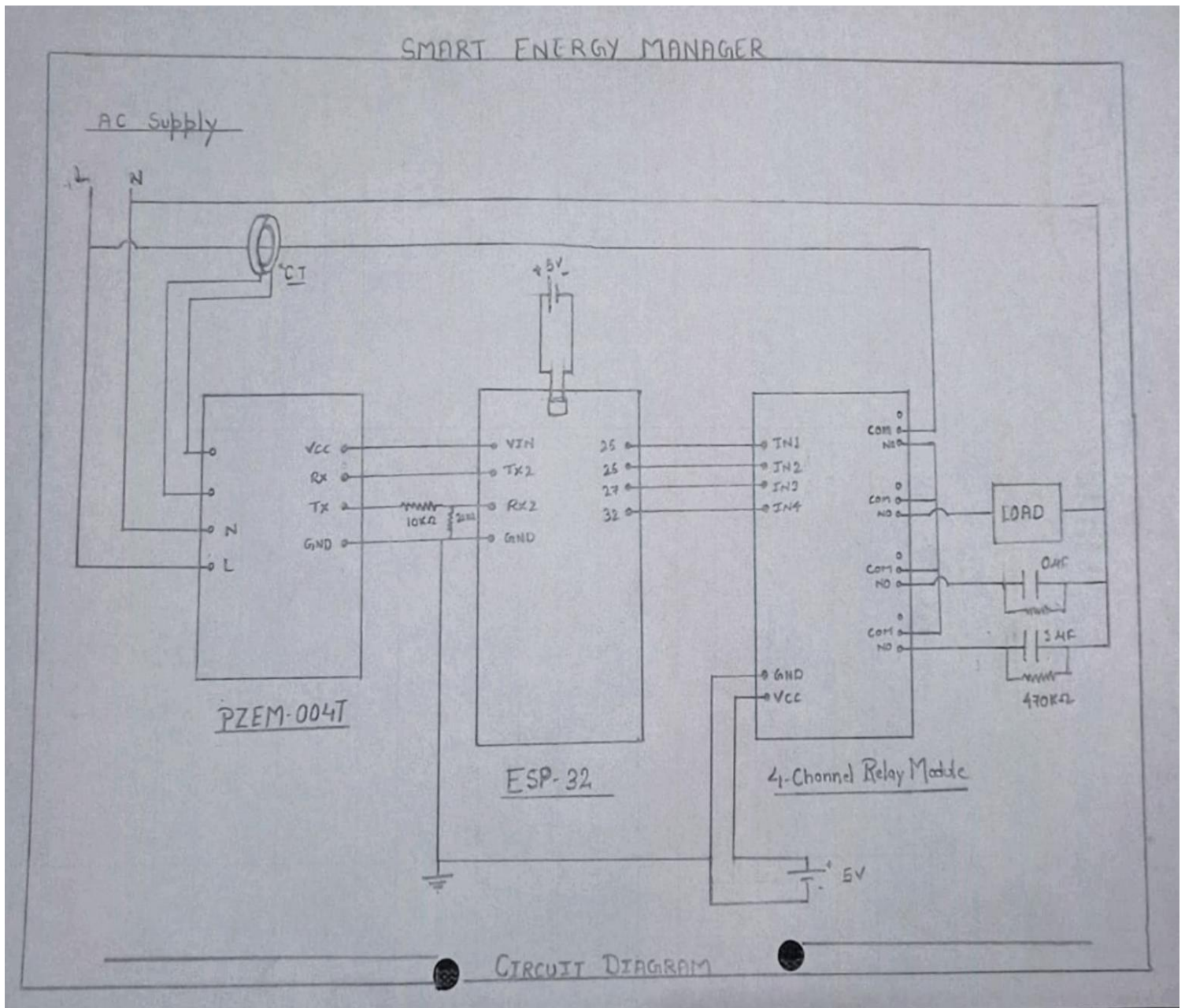


Figure 2: Circuit Diagram of Smart Energy Manager

7. Hardware Requirements



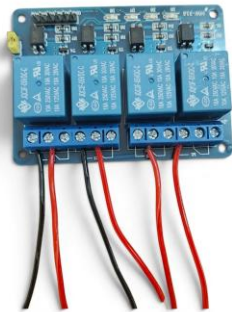
ESP32 Development Board

Main controller with Wi-Fi. It reads data, applies logic, controls relays, and sends values to Blynk.



PZEM-004T Energy Meter Module

Measures AC voltage, current, power, energy, frequency, and power factor.



4-Channel Relay Module

Switches main supply, load supply, and APFC capacitor stages. Active-low relay logic is used.



Current Transformer / CT

Provides isolated current sensing for the PZEM module and improves safety in current measurement.

Other hardware used: capacitor bank for APFC correction, AC demo load, 5 V supply/adaptor for relay and support circuit, terminal blocks, insulated connecting wires, and a mobile phone with internet for Blynk monitoring.

8. Software Requirements

Software tools were used for programming, mobile dashboard design, live monitoring, and alert generation.

Software / Platform	Use in Project
Arduino IDE	Writing, compiling, and uploading the ESP32 program.
ESP32 Board Package	Adds ESP32 board support in Arduino IDE.
Blynk Mobile App and Cloud	Creates dashboard, virtual pins, widgets, controls, graphs, and notifications.
PZEM004Tv30 Library	Reads voltage, current, power, energy, frequency, and PF from PZEM-004T.
BlynkSimpleEsp32 Library	Connects ESP32 to Blynk cloud through Wi-Fi.
Wi-Fi Network	Provides internet connectivity for IoT communication.

The Blynk mobile dashboard is the main display and control interface of this project.

9. Circuit and Pin Connection Details

The ESP32 is connected to the PZEM energy meter through UART communication. The relay module is controlled by ESP32 GPIO pins. The relay module used in the project follows active-low logic, where LOW turns the relay ON and HIGH turns the relay OFF.

Device Pin / Section	ESP32 Pin / Connection	Function
PZEM TX	ESP32 GPIO16 (RXD2)	Serial data from PZEM to ESP32
PZEM RX	ESP32 GPIO17 (TXD2)	Serial command/data from ESP32 to PZEM
Relay IN1	GPIO25	Main supply relay control
Relay IN2	GPIO26	Load relay control
Relay IN3	GPIO27	APFC capacitor stage 1 control
Relay IN4	GPIO32	APFC capacitor stage 2 control
Relay VCC	5 V supply	Relay module power
Relay GND	Common GND	Common ground with ESP32 power side

Safety Note: The AC side of the project must be wired carefully using proper insulation, rated relays/contactors, fuses, and terminal blocks. Testing should be done under faculty/lab supervision.

10. Blynk Mobile App Setup and Interface

The Blynk Mobile dashboard is used as the main monitoring and control interface. It displays electrical readings, trend graphs, billing values, relay status, APFC controls, protection status, and notifications. The Blynk template name used for the project is "Smart Energy Manager".

10.1 Blynk Virtual Pin Mapping

Virtual Pin	Use in Project
V0	Voltage
V1	Current
V2	Power
V3	Power factor
V4	Frequency
V5	Energy consumed
V10	Main supply relay status/control
V11	Load relay status/control
V12	Capacitor stage 1 status/control
V13	Capacitor stage 2 status/control
V20	Emergency stop / kill switch
V21	Over-voltage status
V22	Over-current status
V23	Low power factor status
V30	Energy cost
V31	Energy saved
V32	Money saved
V40	Auto/manual mode
V50	Reset system

10.2 Blynk Interface

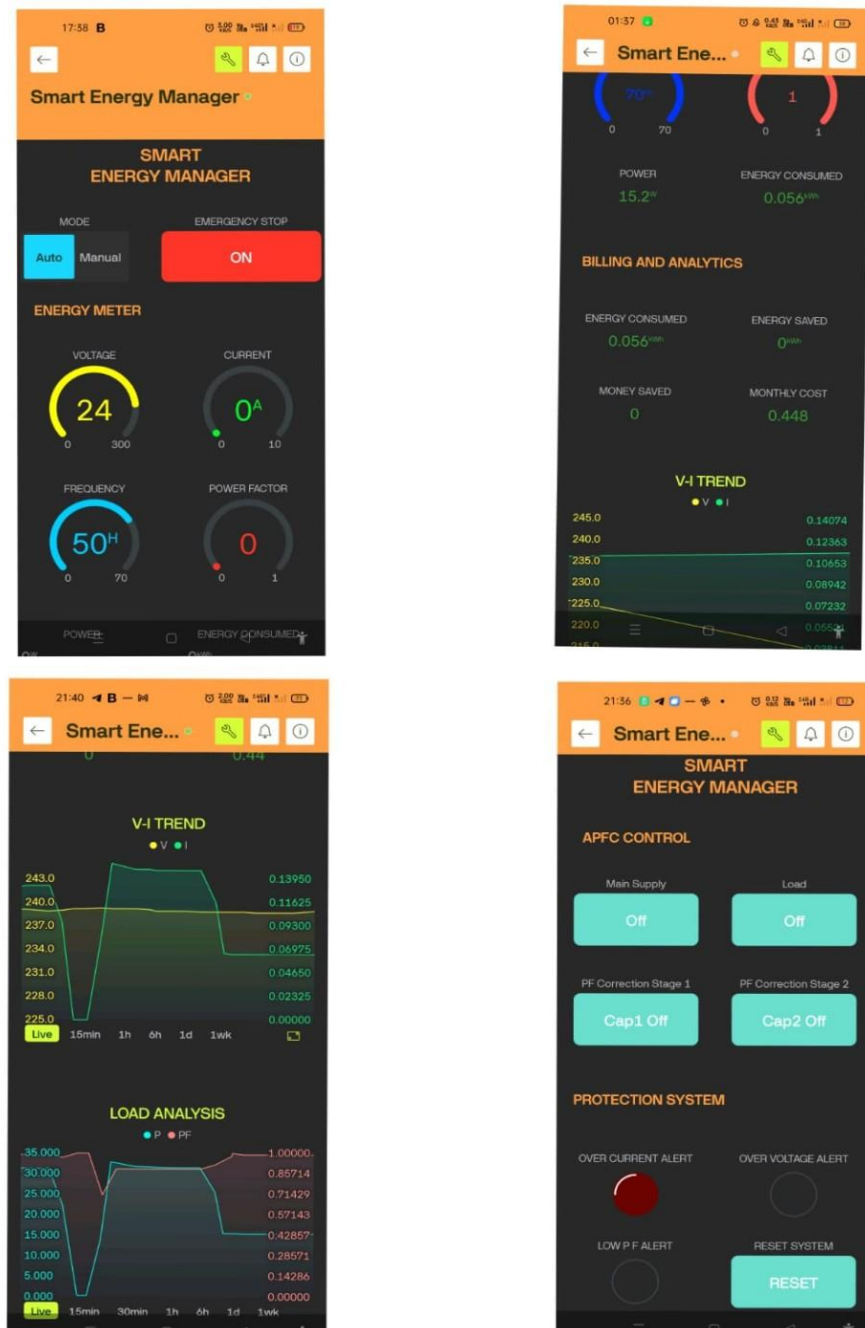


Figure 3: Blynk dashboard overview with energy meter, APFC control, protection and trend screen

10.3 Peak Load Analysis Using Graph

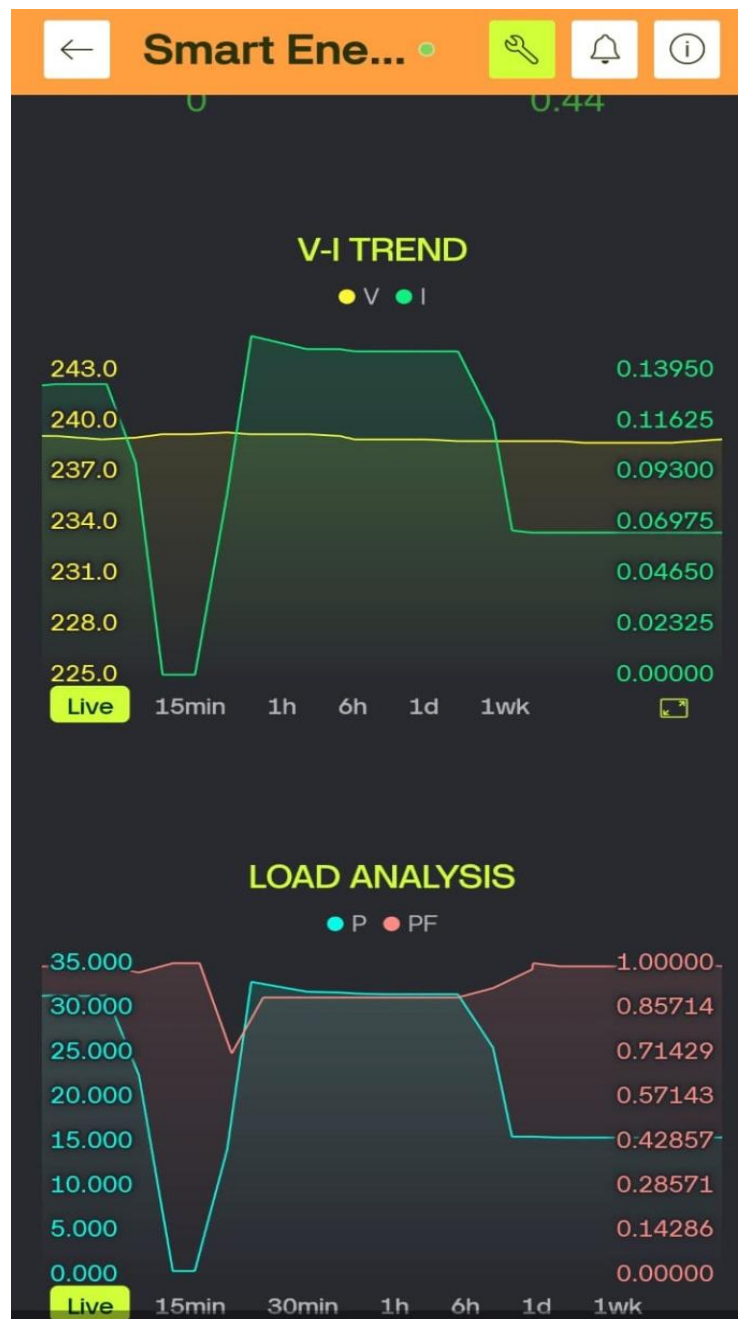


Figure 4: Voltage-current trend and load analysis graph

- This graph is used for observing daily and monthly peak load of the system.

10.4 Monthly Bill Report

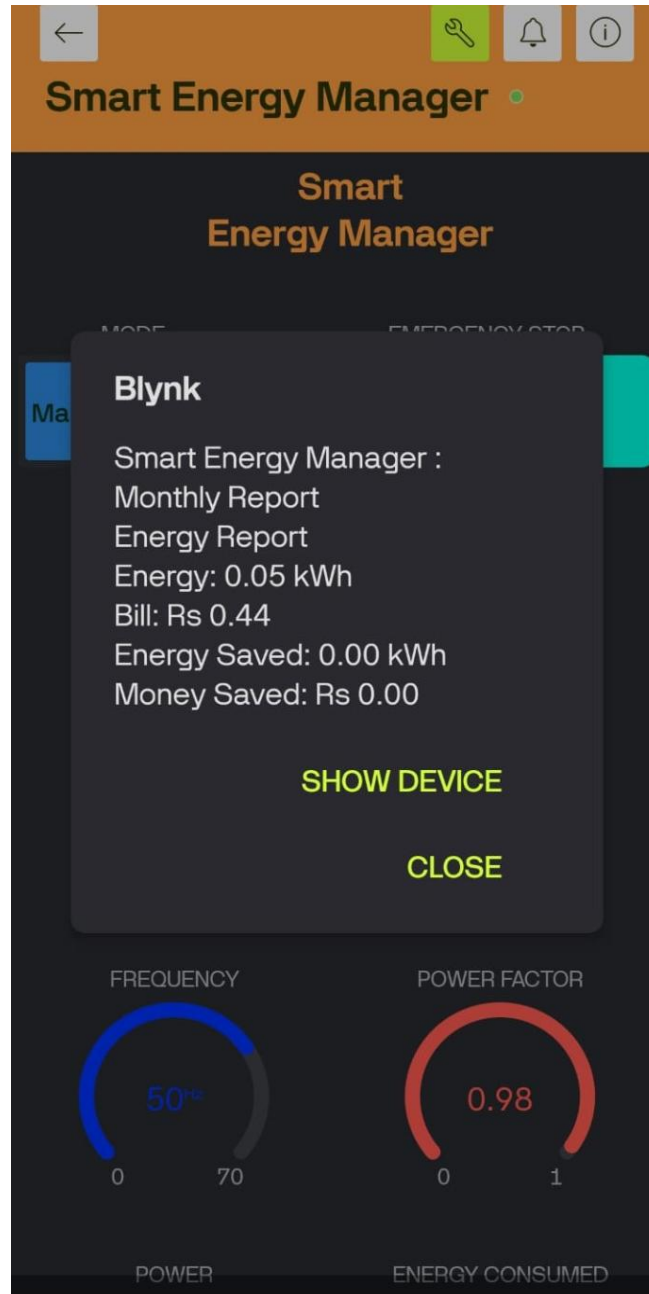


Figure 5: Monthly Bill Report

- This shows energy consumption, estimated bill, energy saved, and money saved. It confirms that the ESP32 code is sending periodic report data to the Blynk application.

10.5 Notifications and Alerts

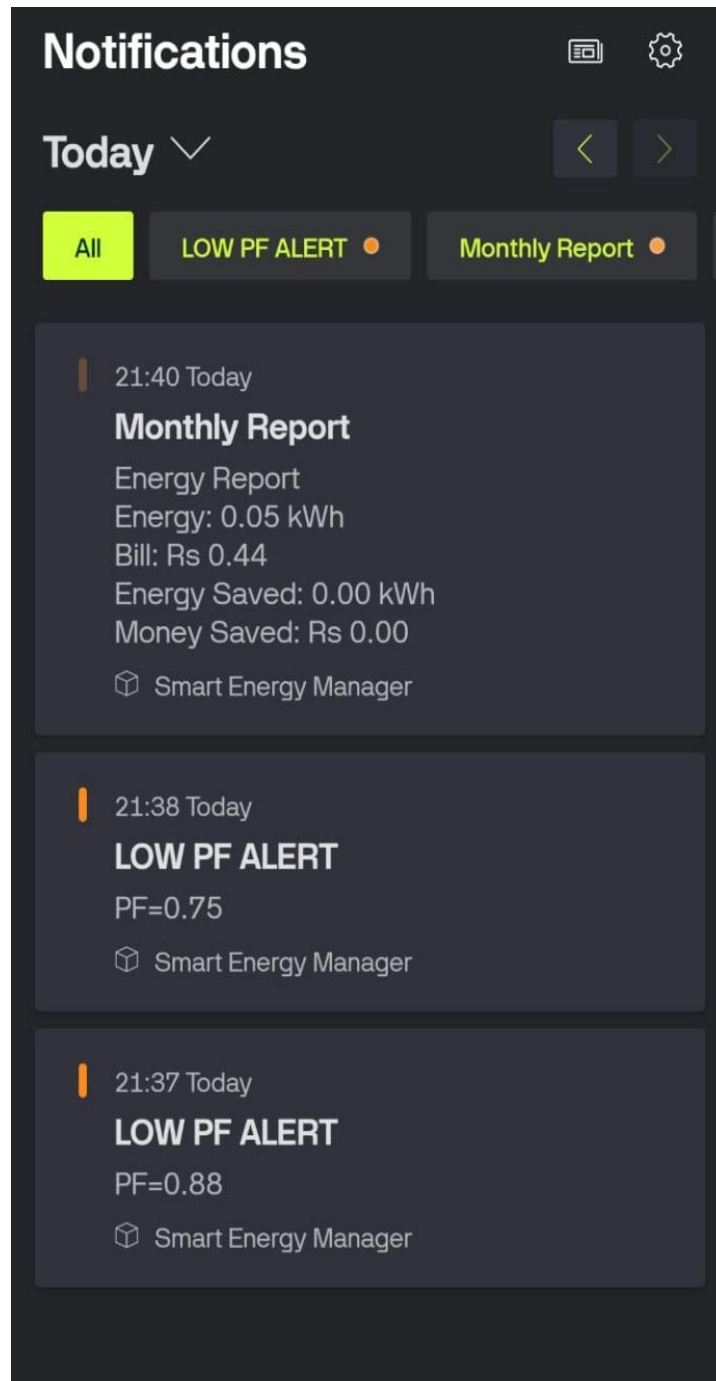


Figure 6: Blynk notification list showing low power factor alerts and energy report

- This shows low power factor alerts generated during testing. PF values such as 0.75 and 0.88 were detected by the system and reported through Blynk. The monthly report notification is also visible in the same event list.

10.6 Live Data Update

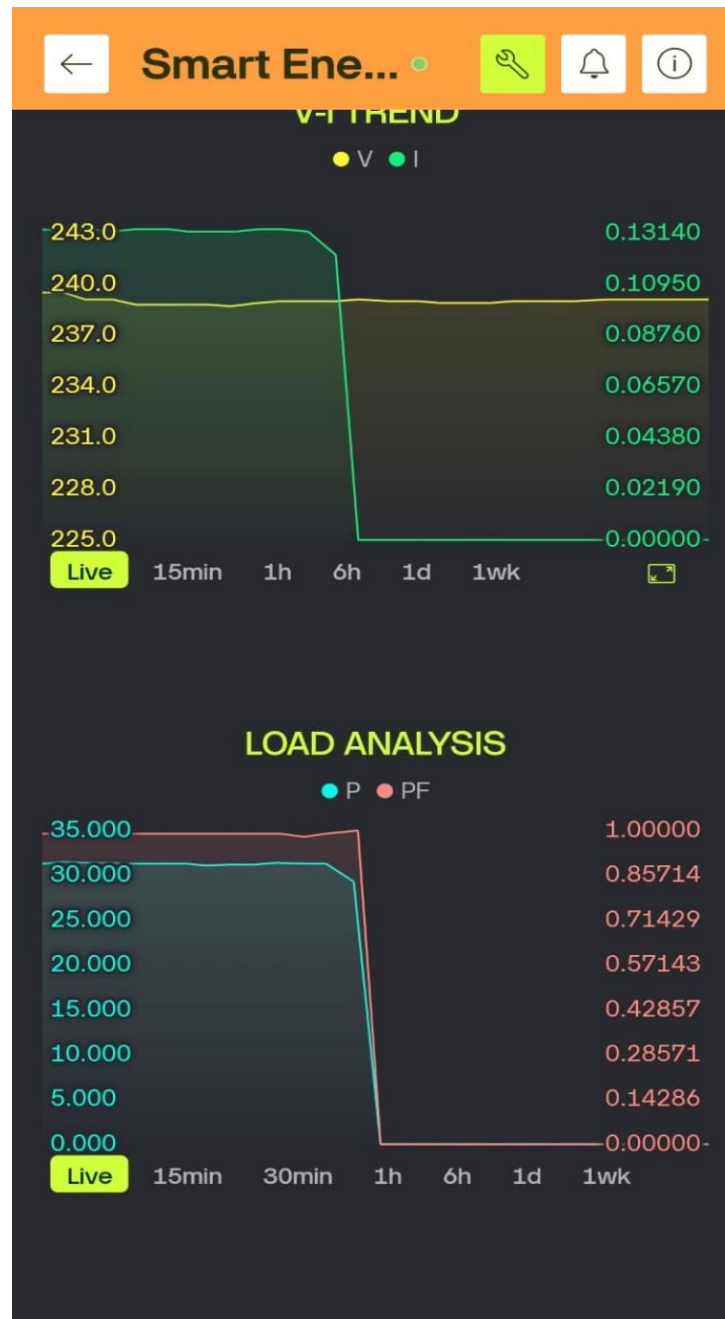


Figure 7: Live trend graph showing voltage, current, power and power factor behavior

- This graph verifies real-time monitoring of electrical parameters. The sudden drop in current and power indicates load disconnection or relay switching during testing. Such graphs help in observing APFC response and load changes over time.

10.7 Graph During Debugging

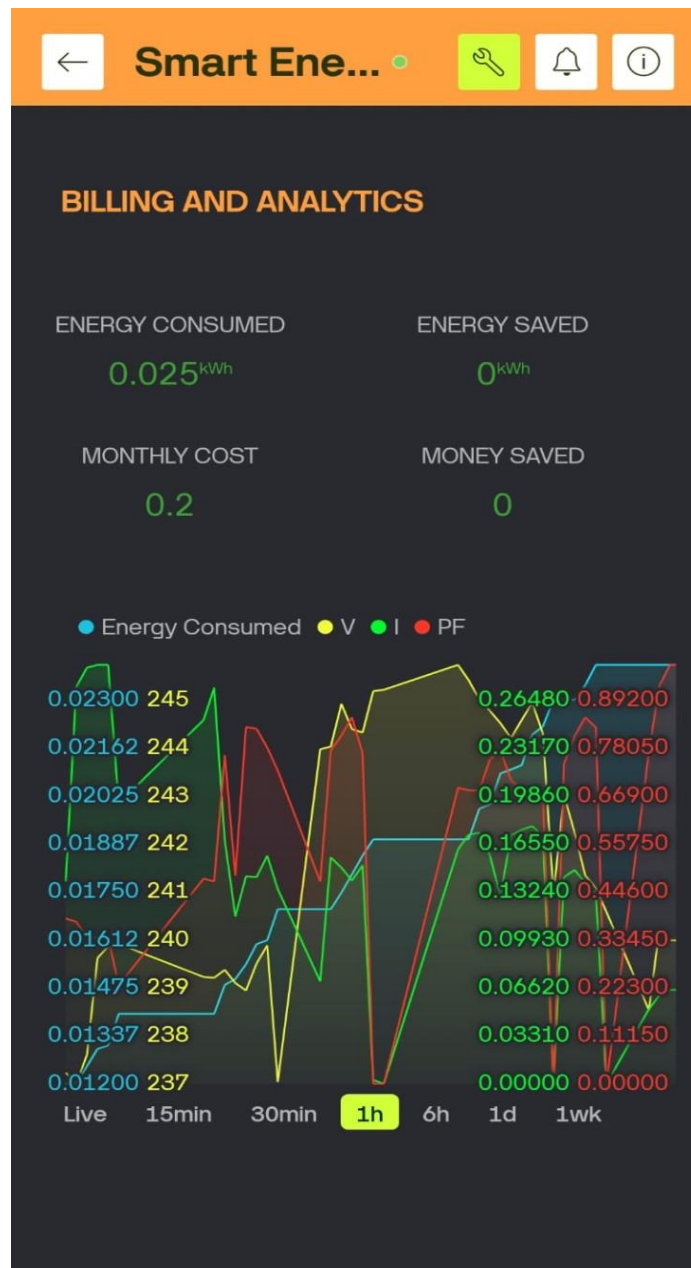


Figure 8: Debugging graph

- This graph combines energy, voltage, current, and power factor data to support result analysis.

11. ESP32 Program Logic

The ESP32 program is divided into different sections for relay control, APFC control, savings calculation, Blynk control, protection, and periodic reporting. The complete cleaned program is included at the end of the report.

11.1 Important Logic Used in the Code

- PZEM sensor readings are taken every second using a Blynk timer.
- Invalid readings are filtered to avoid wrong decisions due to sensor glitches.
- Over-voltage is detected when voltage is greater than 270 V.
- Over-current is detected when current is greater than 1.1 A.
- Faults are latched, so the system remains OFF until reset conditions are safe.
- Low power factor alert is generated when PF is below 0.90 and load is present.
- Capacitor stages are switched gradually with a delay to avoid fast relay switching.
- Energy cost is calculated using a rate of Rs 8 per kWh.
- Periodic energy report is sent to Blynk using event notification.

11.2 Main Protection Thresholds

Parameter	Trip Value	Reset Value
Voltage	270 V	260 V
Current	1.1 A	1.0 A
Power Factor Alert	Below 0.90	Above 0.94 / low load
Capacitor Reset	Above 0.99	Stage-wise removal
Minimum Load	10 W	Below this APFC disabled

12. Testing and Observations

Testing was performed using the Blynk dashboard and live electrical readings obtained from the project. The dashboard screenshots show live monitoring, trend graphs, billing analytics, low power factor alerts, and energy report notifications.

Test Case	Observation / Reading	Result
Normal Monitoring Test	Voltage approx. 239 V, current approx. 0.13 A, power approx. 31.5 W, PF approx. 0.98, frequency 50 Hz	Live parameters were displayed on Blynk successfully.
Low Power Factor Alert Test	PF values such as 0.88 and 0.75 were observed in Blynk notifications	Low PF alert notification was generated successfully.
Energy Report Test	Energy approx. 0.05 kWh and bill approx. Rs 0.44	Periodic energy report notification was generated.
Billing Analytics Test	Energy approx. 0.025 kWh and monthly cost approx. Rs 0.20	Cost calculation was displayed correctly on the dashboard.
Trend Graph Test	Voltage, current, power and PF were plotted over time	Blynk graph showed real-time variation of load parameters.
Emergency Stop / Relay Control	Main supply, load and capacitor controls were available on Blynk	Remote switching and control interface was implemented.
Protection Logic Test	Over-voltage limit 270 V and over-current limit 1.1 A used in code	System is programmed to trip and latch the fault during abnormal conditions.

13. Results and Discussion

- The project successfully monitored voltage, current, power, energy, and power factor.
- Live readings and graphs were shown properly on the Blynk app.
- ESP32 successfully sent sensor data to the Blynk dashboard.
- Capacitor stages turned ON/OFF when low power factor was detected.
- Low power factor alert was received successfully in Blynk notification.
- Energy consumption and bill calculation were shown on the dashboard.
- Relay control system worked properly.
- Over-voltage and over-current protection logic was added in the project.
- Manual restart after fault condition is used for safety purpose.
- Overall, the project successfully works for smart energy monitoring, APFC, and IoT control.

14. Advantages

The Smart Energy Manager provides real-time energy monitoring, automatic power factor correction, and basic load protection in one system. It also gives an easy mobile app-based interface through the Blynk mobile app, which helps the user monitor and control the system remotely.

- It monitors electrical values in **real time**.
- It shows **voltage, current, power, energy, and power factor** on the Blynk app.
- It can be controlled through the blynk mobile app.
- It helps to improve **power factor automatically**.
- It protects the load from **over-voltage and over-current**.
- It has an **emergency stop / kill switch** feature.
- It calculates **energy consumption and estimated bill**.
- It helps in **load analysis**, including daily and monthly peak load determination.
- It provides an easy mobile interface to **monitor and control the system**.
- It sends immediate **notifications and alerts** during fault conditions and disconnects the circuit for safety.

15. Applications

The Smart Energy Manager can be used in small electrical systems where real-time energy monitoring, power factor correction, and load protection are required. It is useful for practical learning, mobile app-based control, and basic energy management applications.

- It can be used in smart home energy monitoring.
- It can be used in small workshops and laboratories.
- It is useful for electrical project demonstrations.
- It can be used for mobile app-based energy monitoring.
- It helps in checking power factor correction practically.
- It can be used for basic load protection.
- It can be used as a prototype for industrial energy management systems.

16. Future Scope

In future, the Smart Energy Manager can be improved for higher load capacity, better safety, and advanced power quality monitoring. The system can also be upgraded for industrial applications by adding data logging, three-phase monitoring, and harmonic analysis.

- The project can be upgraded for **higher load capacity**.
- **Industrial-grade relays/contactors** can be used for better safety.
- **Data logging** can be added to store readings.
- **Three-phase energy monitoring** can be added.
- A **compact PCB** can be designed for the final model.
- **Harmonic monitoring and filtering** can be added to improve power quality.
- A proper **safety enclosure** can be made for practical use.

17. Conclusion

The Smart Energy Manager project was successfully designed as a hardware/software-based system for real-time energy monitoring, automatic power factor correction, load protection and control, and mobile app-based monitoring.

The ESP32 controller, PZEM-004T energy meter, relay module, capacitor bank, and Blynk mobile application were integrated to create a practical working model.

The system measured electrical parameters and displayed them on the Blynk dashboard. It generated low power factor alerts and energy report notifications.

The protection logic provided over-voltage and over-current fault latching, and the reset function allowed restart after safe conditions.

The project meets the objectives of practical execution, testing, and result analysis for an Electrical & Computer Engineering project.

18. References

❑ **ESP32 Development Board Technical Documentation**

Used for understanding ESP32 pin configuration, Wi-Fi connectivity, GPIO control, and microcontroller programming.

❑ **PZEM-004T Energy Meter Module Documentation**

Used for studying voltage, current, power, energy, frequency, and power factor measurement using the PZEM-004T module.

❑ **PZEM004Tv30 Arduino Library Documentation**

Used for interfacing the PZEM-004T energy meter module with ESP32 through serial communication.

❑ **Blynk IoT Platform Documentation**

Used for creating the mobile dashboard, virtual pin mapping, IoT monitoring, and notification system.

❑ **Arduino IDE and ESP32 Board Package Documentation**

Used for writing, compiling, and uploading the program code into the ESP32 development board.

❑ **Relay Module Interfacing Basics**

Used for understanding relay control, active-low relay operation, and load switching through ESP32 GPIO pins.

❑ **Basic Electrical Engineering Concepts**

Used for understanding power factor, energy measurement, relay control, load protection, and capacitor compensation.

❑ **Power Factor Correction and Capacitor Bank Concepts**

Used for studying automatic power factor correction and capacitor stage switching for improving power quality.

19. ESP32 Program Code :

```
/******  
SMART ENERGY MANAGER - FINAL COMPLETE VERSION  
Features:  
- Smart energy meter using PZEM-004T  
- APFC capacitor stage control  
- Kill switch and fault protection  
- Blynk mobile app monitoring and alerts  
- Low power factor alert  
Note: Wi-Fi password and Blynk token are masked for privacy.  
*****/  
  
#define BLYNK_TEMPLATE_ID    "YOUR_TEMPLATE_ID"  
#define BLYNK_TEMPLATE_NAME  "Smart Energy Manager"  
#define BLYNK_AUTH_TOKEN     "YOUR_BLYNK_AUTH_TOKEN"  
  
#include <WiFi.h>  
#include <BlynkSimpleEsp32.h>  
#include <PZEM004Tv30.h>  
  
// Wi-Fi Credentials  
char ssid[] = "YOUR_WIFI_NAME";  
char pass[] = "YOUR_WIFI_PASSWORD";  
  
// PZEM Serial Pins  
#define RXD2 16  
#define TXD2 17  
PZEM004Tv30 pzem(Serial2, RXD2, TXD2);  
  
// Relay Pins  
#define R1 25  
#define R2 26  
#define R3 27  
#define R4 32  
  
#define RELAY_ON LOW  
#define RELAY_OFF HIGH  
  
BlynkTimer timer;  
  
// Protection Limits  
float MAX_V = 270, RESET_V = 260;  
float MAX_I = 1.1, RESET_I = 1.0;  
float PF_LOW = 0.90;  
float PF_ALERT_RESET = 0.94;  
float PF_CAP_RESET = 0.99;  
float MIN_LOAD = 10;  
  
// Cost and Saving  
float ENERGY_RATE = 8.0;  
float totalEnergySaved = 0;  
float totalMoneySaved = 0;  
float lineResistance = 1.0;  
float currentBeforeAPFC = 0;  
float currentAfterAPFC = 0;  
unsigned long lastSaveTime = 0;  
unsigned long lastReportTime = 0;  
  
// System States  
bool autoMode = true;  
bool killSwitch = false;  
bool faultLatched = false;  
bool ovFault = false;  
bool ocFault = false;  
bool lowPFFault = false;  
bool pfNotified = false;  
  
// Notification Flags  
bool ovNotified = false;  
bool ocNotified = false;
```

```

// Relay States
bool r1 = true, r2 = true, r3 = false, r4 = false;
int capStage = 0;

unsigned long lastCap = 0;
unsigned long CAP_DELAY = 5000;
unsigned long startupTime = 0;
bool systemReady = false;

// Readings
float V, I, P, PF, F, E;

// ----- RELAY CONTROL -----
void setRelay(int pin, bool st) {
    digitalWrite(pin, st ? RELAY_ON : RELAY_OFF);
}

void applyRelays() {
    if (killSwitch || faultLatched) {
        r1 = false;
        r2 = false;
        r3 = false;
        r4 = false;
        capStage = 0;
    }

    if (!r1 || !r2) {
        r3 = false;
        r4 = false;
        capStage = 0;
    }

    setRelay(R1, r1);
    setRelay(R2, r2);
    setRelay(R3, r3);
    setRelay(R4, r4);

    Blynk.virtualWrite(V10, r1);
    Blynk.virtualWrite(V11, r2);
    Blynk.virtualWrite(V12, r3);
    Blynk.virtualWrite(V13, r4);
}

void setCapStage(int s) {
    capStage = s;
    r3 = (s == 1 || s == 3);
    r4 = (s == 2 || s == 3);
    lastCap = millis();
}

// ----- APFC CONTROL -----
void apfc() {
    if (P < MIN_LOAD || I < 0.03) {
        setCapStage(0);
        return;
    }

    if (PF > PF_CAP_RESET) {
        if (capStage == 3 && millis() - lastCap > CAP_DELAY) {
            setCapStage(2);
            return;
        }
        if (capStage == 2 && millis() - lastCap > CAP_DELAY) {
            setCapStage(1);
            return;
        }
        if (capStage == 1 && millis() - lastCap > CAP_DELAY) {
            setCapStage(0);
            return;
        }
    }

    if (PF < PF_LOW) {
        if (capStage == 0) {
            currentBeforeAPFC = I;
            setCapStage(1);
        }
    }
}

```

```

        return;
    }
    if (capStage == 1 && millis() - lastCap > CAP_DELAY) {
        setCapStage(2);
        return;
    }
    if (capStage == 2 && millis() - lastCap > CAP_DELAY) {
        setCapStage(3);
        return;
    }
}
}

// ----- SAVINGS CALCULATION -----
void updateSavings() {
    unsigned long now = millis();

    if (lastSaveTime == 0) {
        lastSaveTime = now;
        return;
    }

    float hours = (now - lastSaveTime) / 3600000.0;
    lastSaveTime = now;

    if ((r3 || r4) && P > MIN_LOAD) {
        currentAfterAPFC = I;

        float lossBefore = currentBeforeAPFC * currentBeforeAPFC * lineResistance;
        float lossAfter = currentAfterAPFC * currentAfterAPFC * lineResistance;
        float powerSaved = lossBefore - lossAfter;

        if (powerSaved < 0) {
            powerSaved = 0;
        }

        float energySaved = (powerSaved * hours) / 1000.0;
        totalEnergySaved += energySaved;
        totalMoneySaved = totalEnergySaved * ENERGY_RATE;
    }

    Blynk.virtualWrite(V31, totalEnergySaved);
    Blynk.virtualWrite(V32, totalMoneySaved);
}

// ----- MAIN TASK -----
void mainTask() {
    if (millis() - startupTime > 2000) {
        systemReady = true;
    }

    V = pzem.voltage();
    I = pzem.current();
    P = pzem.power();
    PF = pzem.pf();
    F = pzem.frequency();
    E = pzem.energy();

    if (isnan(V) || isnan(I) || isnan(P) || isnan(PF)) return;

    // Invalid reading filter
    if (V < 50 || V > 300) return;
    if (I < 0 || I > 20) return;
    if (P < 0 || P > 5000) return;

    // Power factor glitch correction
    if (PF < 0 || PF > 1) {
        PF = 0;
    }

    // Over-voltage protection
    if (V > MAX_V) {
        ovFault = true;
        faultLatched = true;
        if (!ovNotified) {
            Blynk.logEvent("over_voltage", "Voltage=" + String(V));
            ovNotified = true;
        }
    }
}

```

```

    }
}

// Over-current protection
if (I > MAX_I) {
    ocFault = true;
    faultLatched = true;
    if (!ocNotified) {
        Blynk.logEvent("over_current", "Current=" + String(I));
        ocNotified = true;
    }
}

// Low power factor alert
if (systemReady && PF < PF_LOW && P > MIN_LOAD && I > 0.03) {
    lowPFFault = true;
    if (!pfNotified) {
        Blynk.logEvent("low_power_factor", "PF=" + String(PF));
        pfNotified = true;
    }
}

if (PF > PF_ALERT_RESET || P < MIN_LOAD) {
    lowPFFault = false;
    pfNotified = false;
}

// Send live data to Blynk
Blynk.virtualWrite(V0, V);
Blynk.virtualWrite(V1, I);
Blynk.virtualWrite(V2, P);
Blynk.virtualWrite(V3, PF);
Blynk.virtualWrite(V4, E);
Blynk.virtualWrite(V5, E);

float cost = E * ENERGY_RATE;
Blynk.virtualWrite(V30, cost);
Blynk.virtualWrite(V21, ovFault ? 1 : 0);
Blynk.virtualWrite(V22, ocFault ? 1 : 0);
Blynk.virtualWrite(V23, lowPFFault ? 1 : 0);

if (autoMode && !faultLatched && !killSwitch) {
    apfc();
}

applyRelays();
updateSavings();

// Periodic energy report notification
if (millis() - lastReportTime >= 180000) {
    String report =
        "Energy Report\n" +
        String("Energy: ") + String(E, 2) + " kWh\n" +
        String("Bill: Rs ") + String(E * ENERGY_RATE, 2) + "\n" +
        String("Energy Saved: ") + String(totalEnergySaved, 2) + " kWh\n" +
        String("Money Saved: Rs ") + String(totalMoneySaved, 2);

    Blynk.logEvent("monthly_report", report);
    lastReportTime = millis();
}
}

// ----- BLYNK CONTROLS -----
BLYNK_WRITE(V20) {
    killSwitch = param.asInt();
    applyRelays();
}

BLYNK_WRITE(V40) {
    autoMode = param.asInt();
}

// Manual mode relay controls
BLYNK_WRITE(V10) {
    if (!autoMode) {
        r1 = param.asInt();
        applyRelays();
    }
}

```

```

    }
}

BLYNK_WRITE(V11) {
  if (!autoMode) {
    r2 = param.asInt();
    applyRelays();
  }
}

BLYNK_WRITE(V12) {
  if (!autoMode) {
    r3 = param.asInt();
    applyRelays();
  }
}

BLYNK_WRITE(V13) {
  if (!autoMode) {
    r4 = param.asInt();
    applyRelays();
  }
}

// Manual restart after fault
BLYNK_WRITE(V50) {
  if (param.asInt() == 1) {
    if (V < RESET_V && I < RESET_I) {
      faultLatched = false;
      ovFault = false;
      ocFault = false;
      ovNotified = false;
      ocNotified = false;
      r1 = true;
      r2 = true;
    }
  }
  applyRelays();
}

// ----- SETUP -----
void setup() {
  Serial.begin(115200);

  pinMode(R1, OUTPUT);
  pinMode(R2, OUTPUT);
  pinMode(R3, OUTPUT);
  pinMode(R4, OUTPUT);

  setRelay(R1, false);
  setRelay(R2, false);
  setRelay(R3, false);
  setRelay(R4, false);

  startupTime = millis();

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  autoMode = true;
  Blynk.virtualWrite(V40, 1);

  timer.setInterval(1000L, mainTask);
}

void loop() {
  Blynk.run();
  timer.run();
}

```